AIDLC / ENGINEERING PRACTICE

AIDLC, and Why Spec Driven Development Is the Centre of It

Agile narrowed the gap between intent and code by shipping faster. AIDLC closes it differently. It makes intent the artifact, and the build a rendering of it.

• Gaurav Arora • 8 minute read • May 2026

IN THIS PIECE

What AIDLC Is

AI Driven Development Lifecycle restructures every phase of delivery (requirements through operations) around the assumption that AI is a primary participant, not a side tool. It does not make Agile faster. It changes what the work is. Less time on keystrokes, more on intent.

Spec Driven Development sits at the heart of it. Governance, tooling, and role design all flow from the spec.

Spotlight on Spec Driven Development

A spec is a living markdown file in the repository. Goal, constraints, architectural decisions, task list. One rule. Nothing gets built until the spec exists. The AI reads it before doing anything. Clear spec, clear build. Vague spec, and the vagueness surfaces on day one instead of month six.

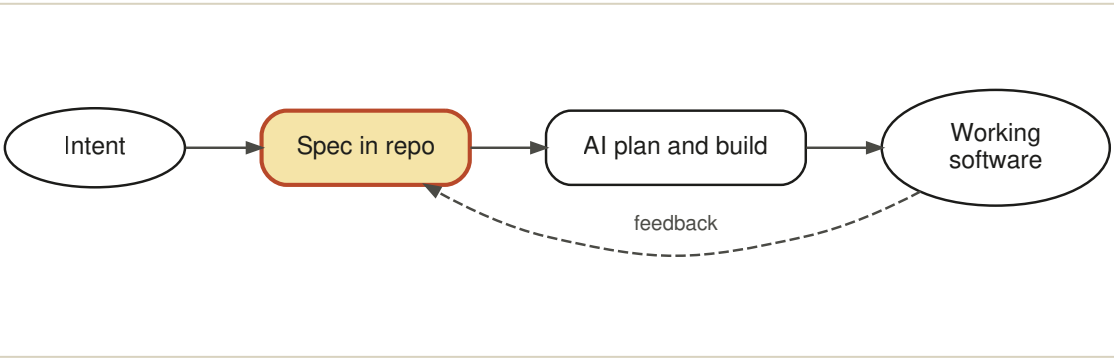


Figure 1. The spec is the contract. The code is a rendering of it.

> *If you cannot write down what you are building in plain language, you do not know what you are building.*

Agile vs AIDLC: The Process

The shapes are different. Agile loops around the ticket. AIDLC loops around the spec.

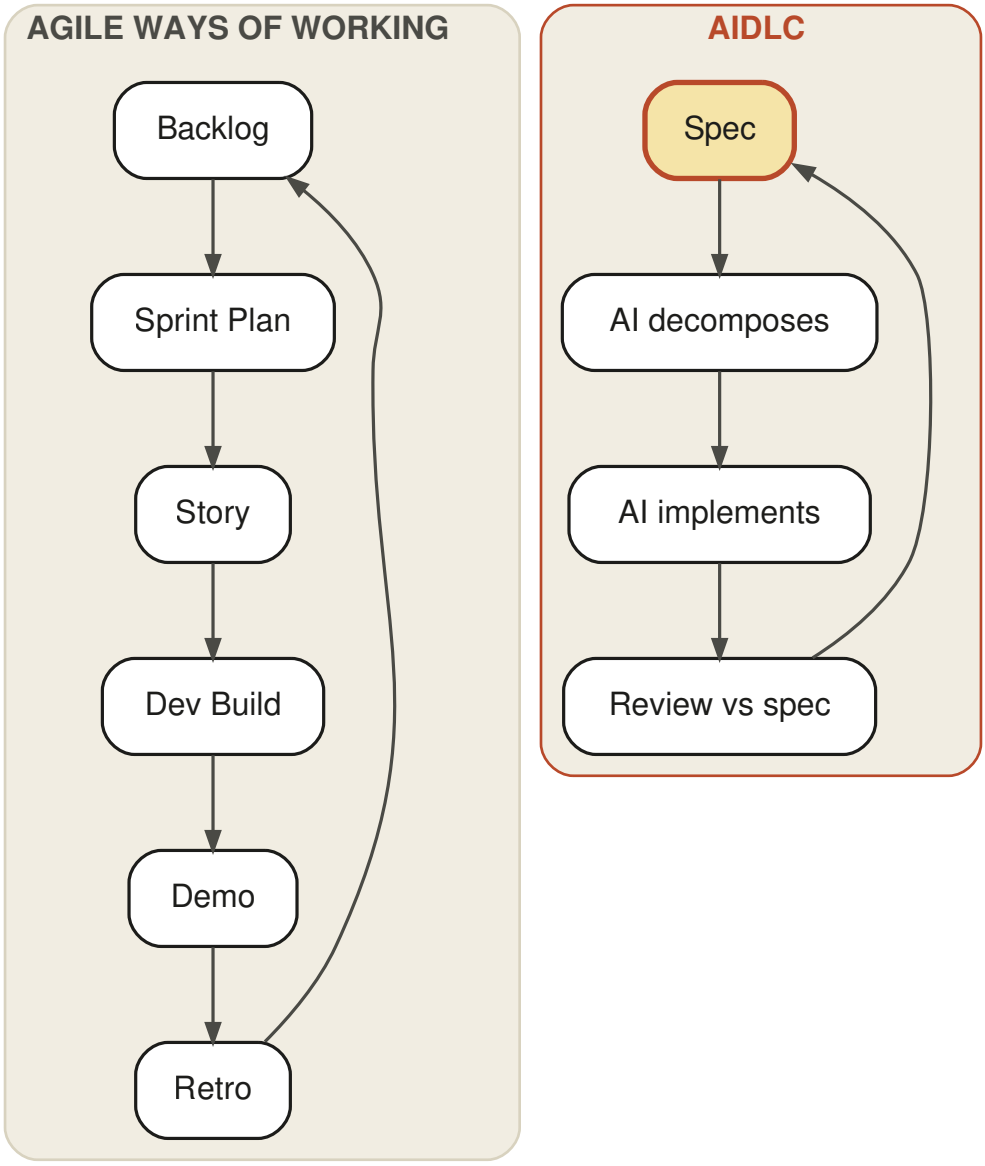


Figure 2. Agile cycles through tickets. AIDLC cycles through the spec.

PARAMETER	AGILE WAYS OF WORKING	AIDLC
Primary artifact	User story in a tracker	Spec file in the repository
Source of truth	Ticket descriptions plus tribal knowledge	Versioned spec alongside code
Planning cadence	Two week sprint	Continuous, triggered by spec change
Estimation	Story points assigned by developer	Task graph generated from spec
Build owner	Developer	AI builds, human reviews
Quality gate	Peer code review	Spec adherence check, then code review
Knowledge persistence	Lives in heads, Slack, ticket history	Lives in the spec, survives turnover
Cost of change	Reopen ticket, replan sprint	Edit the spec, regenerate the build
Onboarding	Weeks of context transfer	Read the spec

Stakeholder and Mindset Shift

This is where most adoption efforts stall. Tooling is the easy part. The mental model is not.

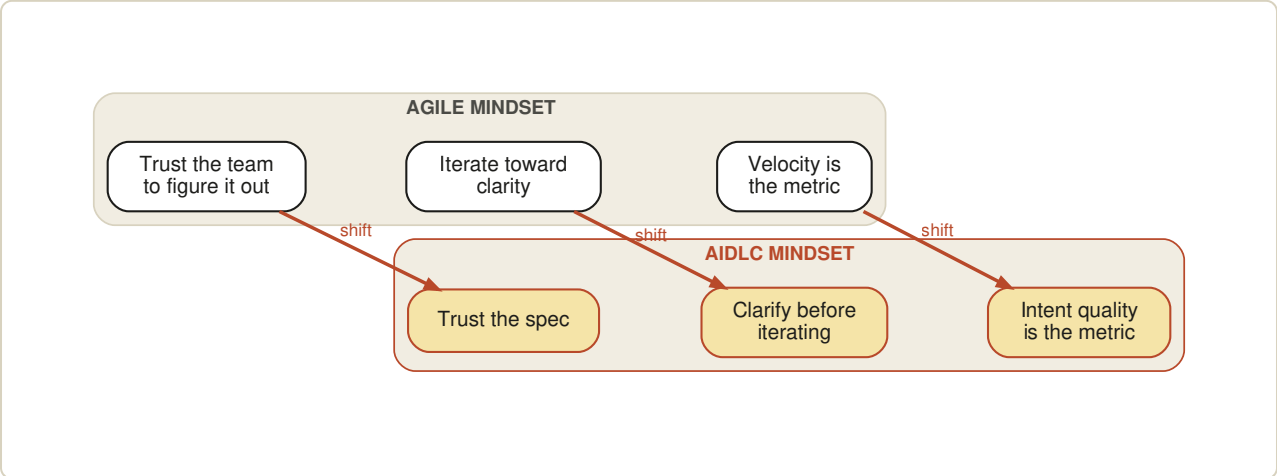


Figure 3. Three beliefs that have to be updated for AIDLC to work.

PARAMETER	AGILE STAKEHOLDER	AIDLC STAKEHOLDER
Role in planning	Approves epics, prioritises the backlog	Co authors the spec
Engagement cadence	Sprint review every two weeks	At every meaningful spec change
Tolerance for ambiguity	High, "we will figure it out in sprint"	Low, ambiguity blocks the build
Skill required	Storytelling, prioritisation, negotiation	Precise written articulation
Common failure mode	Builds the wrong thing efficiently	Cannot start until intent is real
Where time is spent	Standups, refinements, demos	Writing and reviewing specs

Where Behaviour Has to Change

Left side is current Agile behaviour. Right side is what AIDLC requires.

TODAY (AGILE)		AIDLC
~~Verbal requirements in refinement~~	→	Written spec sections owned by stakeholder
~~Accept vague acceptance criteria~~	→	Define done before build starts
~~Reprioritise mid sprint via Slack~~	→	Edit the spec, regenerate the plan
~~Sign off at demo day~~	→	Sign off at spec, demo is verification
~~Escalate scope creep in retro~~	→	Scope drift visible in spec diff
~~PM writes tickets, dev writes code~~	→	Both write spec, AI writes code

Three places stakeholders push back. Writing things down feels slow when nothing is being built yet. It is not slow. It is the build, in a different medium. The spec also surfaces disagreement that Agile ceremonies usually paper over with optimism. That surfacing is the value, not a defect. Finally, ownership shifts. The product manager who used to approve outcomes now co authors the input. Uncomfortable for a month, then it is not.

What Engineering Has to Change

PARAMETER	AGILE ENGINEERING	AIDLC ENGINEERING
Definition of done	Code merged, tests pass	Spec satisfied, code merged, tests pass
Code ownership	Module or service owners	Spec owners
Review focus	Diff against previous code	Diff against spec
Architectural decisions	Captured in ADRs after the fact	Captured in the spec before code
Pairing model	Two humans on one keyboard	One human, one AI, one spec
Refactoring trigger	Tech debt review	Spec change

A Concrete Flow

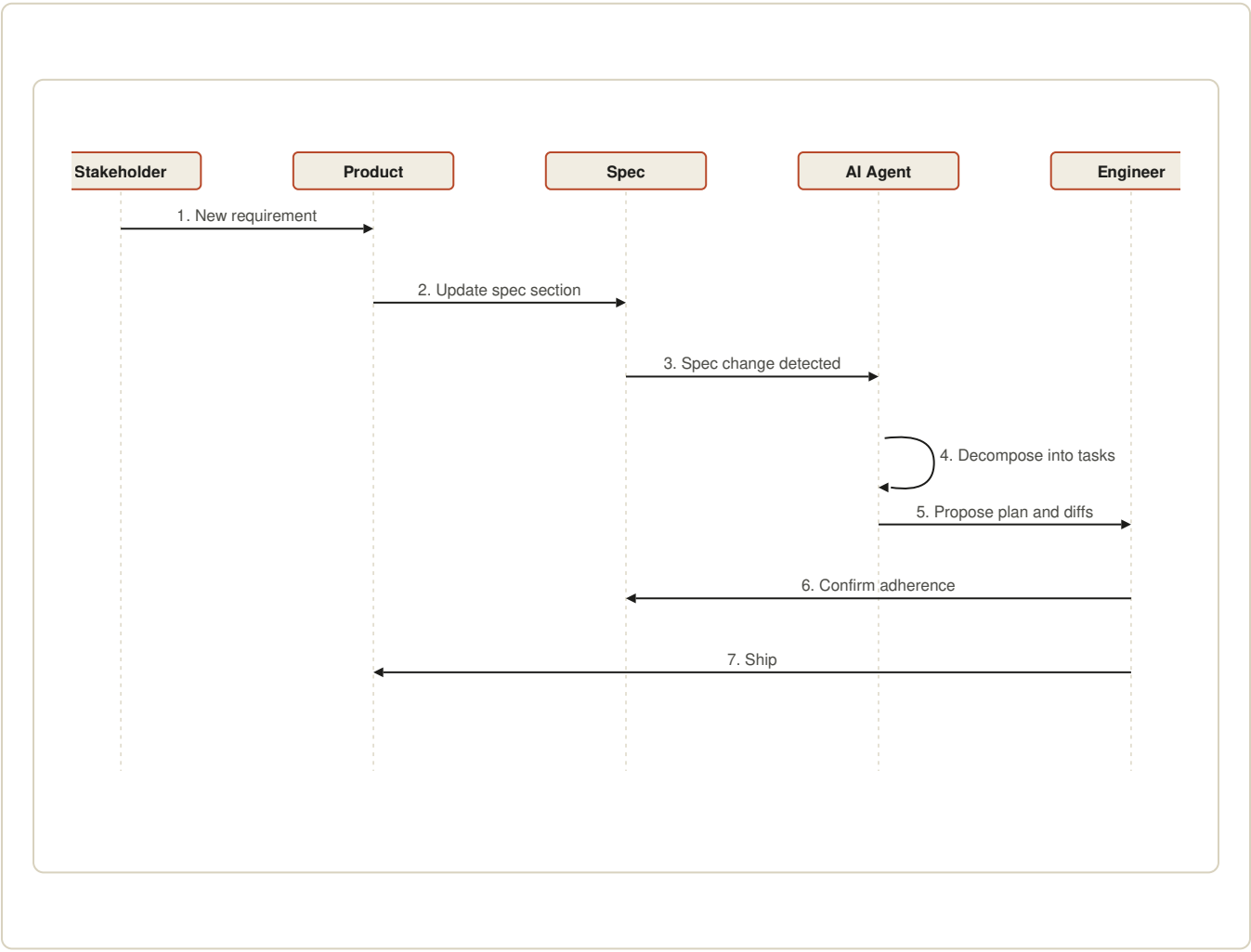


Figure 4. The work moves through the spec, not through a standup.

The Tooling Reality

The methodology is tool agnostic. Execution is not. A spec only works if the AI doing the build can read it, reason about it, and flag drift from it.

PARAMETER	AGILE TOOLING TODAY	AIDLC TOOLING
Where intent lives	Jira, Confluence, Slack	Spec file in the repo
Where code lives	Git	Git, next to the spec
How they stay in sync	Manual updates, drift is constant	Spec is the source, build follows
Audit trail	Ticket history	Spec version history

PLATFORM FIT

TOOL	SPEC MECHANISM	PRACTICAL LIMIT
Claude Code	CLAUDE.md plus memory plus task system	Spec structure is convention, not enforced
Cursor	Rules files in `.cursor/rules/`	Read only, no task tracking, AI cannot revise the spec
Kiro	Native specs in `.kiro/specs/` with hook enforcement	Heavy for exploratory work where the spec is still forming
Codex / GPT	No native spec layer	Stateless, methodology lives in user discipline
Gemini Gems	Persona prompts, not repository aware	Spec and code drift silently

EMERGING FRAMEWORK LANDSCAPE

FRAMEWORK	WHAT IT STANDARDISES	WHERE IT FITS
Kiro spec-task-hook	Spec, generated tasks, file save enforcement	Greenfield product squads
Anthropic SKILL.md	Reusable spec fragments invoked by agents	Capability libraries shared across projects
OpenAPI plus JSON Schema	Interface contracts as machine readable spec	Service boundaries and API design
Given / When / Then	Verification block format	Behavioural acceptance, regulated work
ADR plus Spec	Decision records embedded in living spec	Architecture heavy programmes

---

Implementing SDD in an Organisation

SDD does not arrive through a memo. One squad runs it, proves the loop, exports the pattern. Start with an honest audit of where intent lives today and how often it gets lost.

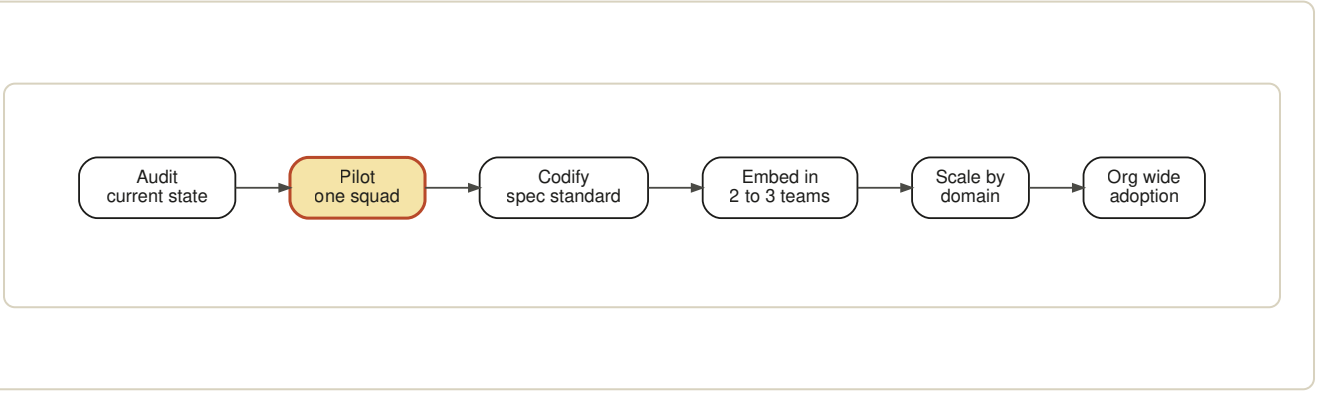


Figure 5. Phased rollout. Skip a stage and the next one fails.

AUDIT DIMENSIONS

PARAMETER	WHAT AGILE AUDITS LOOK FOR	WHAT AIDLC AUDITS LOOK FOR
Requirements	User stories with acceptance criteria in Jira	Spec file present per service, version controlled
Definition of done	Tests pass, PR approved	Spec satisfied, tests pass, spec diff reviewed
Change control	Change request workflow	Spec diff is the change request
Governance	Sprint review, steerco	Spec review board, architecture council
Compliance trace	Ticket history exported on demand	Git history of the spec is the audit log

ROLLOUT PLAN BY QUARTER

QUARTER	FOCUS	EXIT CRITERIA
Q1	Audit, pick pilot, define spec template	One squad shipping through specs, template signed off
Q2	Pilot runs in parallel with Agile, measure both	Cycle time and rework data captured for comparison
Q3	Codify standards, train second cohort	Three squads on SDD, spec linter in CI
Q4	Scale across one domain, retire ceremonies that no longer add value	Whole domain on SDD, Jira role reduced to coordination only

Three risks during rollout. Cultural resistance from teams who equate writing with bureaucracy. A spec literacy gap that no one wants to admit. Governance lag, where audit and risk functions still demand artefacts the new process does not produce. Surface all three in the audit. None of them surprise you later if you name them now.

The Product Owner Shift

Product owners spend their day inside Jira. Stories, epics, acceptance criteria, refinements, standups. AIDLC pulls almost all of that work into the spec. The PO does not disappear. The work moves.

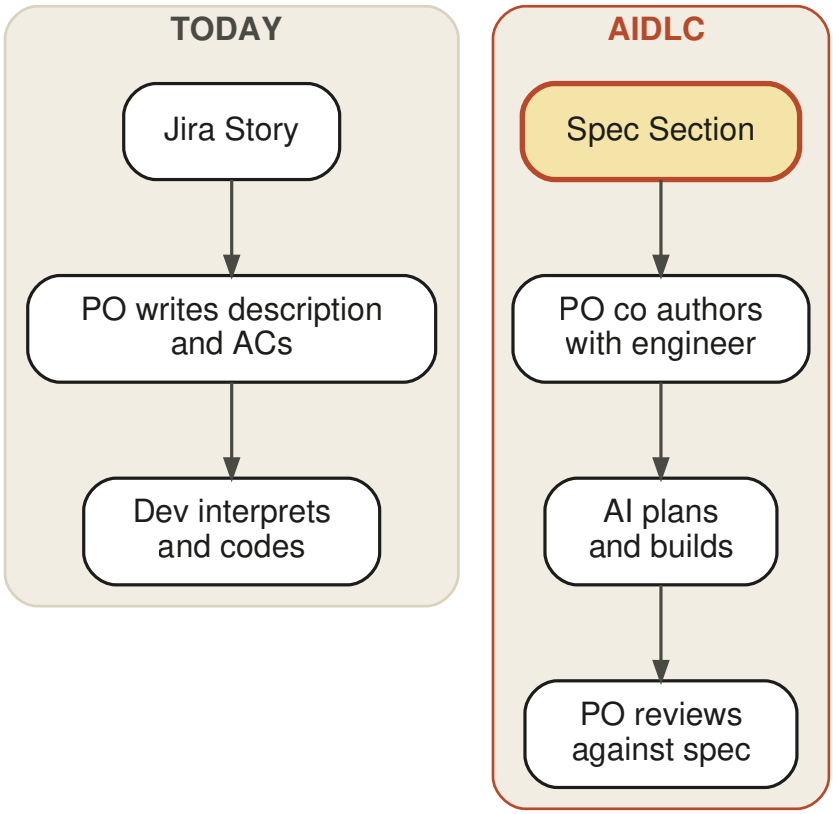


Figure 6. The PO moves from describing work to defining intent.

PARAMETER	PRODUCT OWNER IN AGILE	PRODUCT OWNER IN AIDLC
Primary tool	Jira	Spec file in the repository
Output	Story with acceptance criteria	Spec section with intent, constraints, examples
Cadence	Daily standup, weekly refinement	Spec authoring sessions, async review
Core skill	Storytelling, prioritisation	Precise written articulation
Success measure	Velocity, throughput	Spec clarity, build acceptance rate
Time allocation	60% in meetings	60% writing and reviewing specs
Coverage	One or two squads	One or two domains

JIRA TO SPEC MAPPING

JIRA CONCEPT	MAPS TO IN SDD	NOTES
Epic	Spec module	Top level outcome and constraints
Story	Spec task block	Single behaviour, owned by one spec section
Acceptance criteria	Verification block inside the spec	Given, when, then format works well
Bug	Spec gap or implementation drift	Fix the spec first if intent was wrong
Standup update	Spec diff comment	Async, visible, traceable

> *The PO who used to approve outcomes now co authors the input. Different job, same goal.*

Frameworks and Patterns

SDD is still consolidating. The patterns below show up across the teams that have made it work for more than a quarter. Treat them as building blocks, not a rulebook.

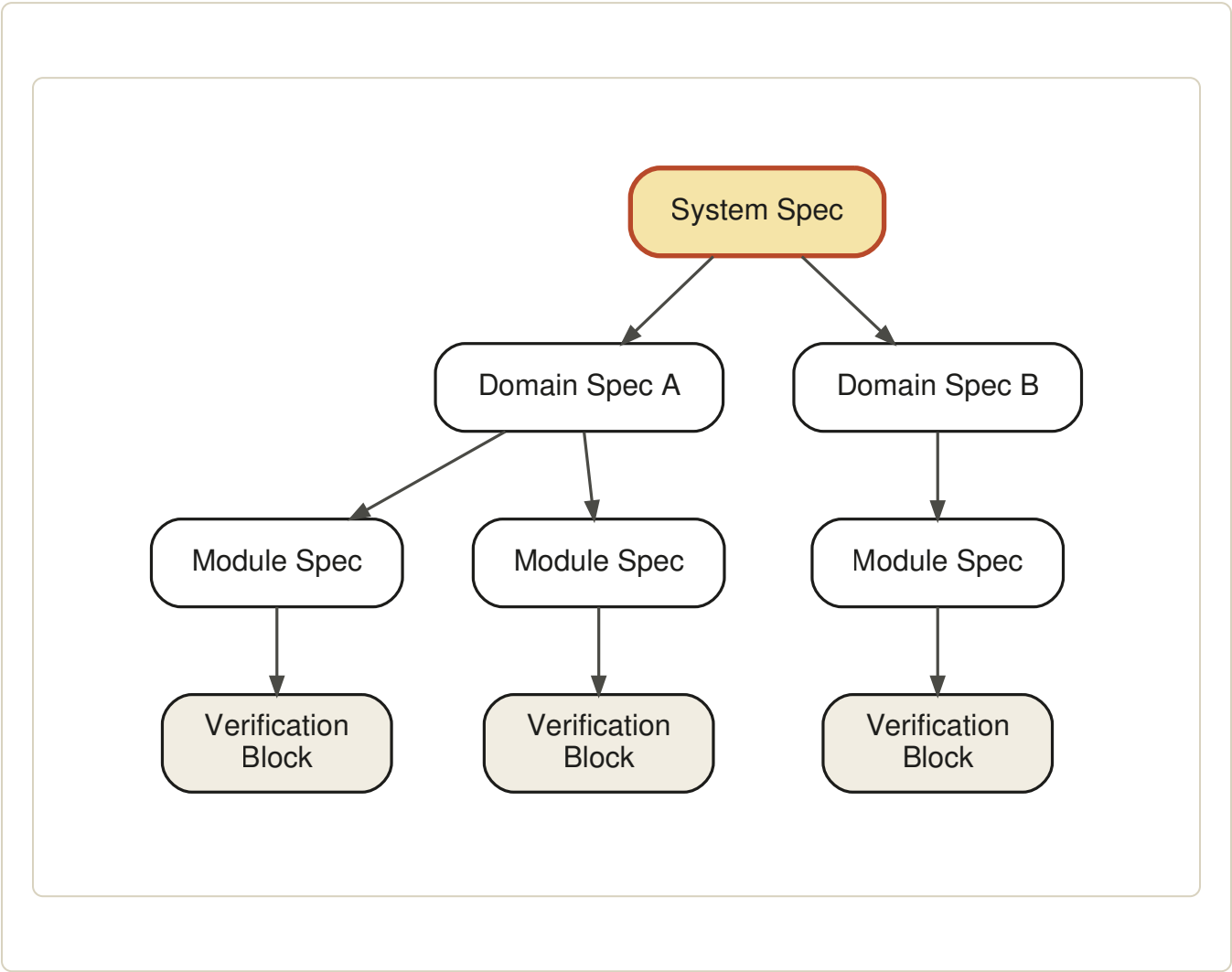


Figure 7. Hierarchical spec structure. System level intent flows down. Verification flows up.

PATTERN CATALOGUE

PATTERN	WHEN TO USE	COMMON PITFALL
Spec First	All new builds	Treating it as optional under deadline pressure
Spec as ADR	Architectural choice required	Burying the decision in prose with no rationale
Living Spec	Long lived services	Spec rot when the team rotates and no one owns it
Hierarchical Spec	Large systems, multiple domains	Over fragmenting and losing the system view
Verification Block	Anywhere correctness matters	Vague pass criteria that read like marketing copy
Spec Diff Review	High change velocity teams	Reviewing code PRs without checking spec changes

---

Learning Curve and Maturity

Different curve for every role. Treat it as a capability ladder, not a course. Five levels of fluency, twelve weeks to get a cohort from Level 1 to Level 3.

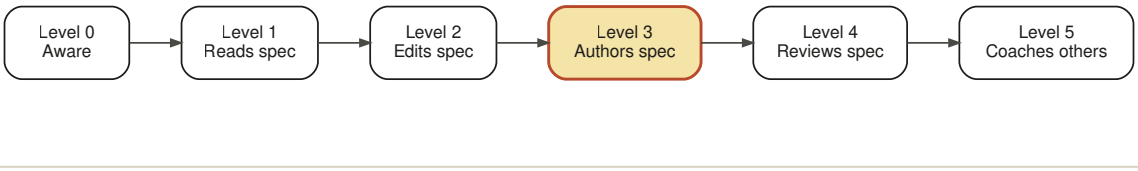


Figure 8. The maturity ladder. Most stakeholders need to reach Level 3 for the model to compound.

ITERATIVE TRAINING PLAN, 12 WEEKS

WEEKS	FOCUS	EXERCISE
1 to 2	Read existing specs	Annotate three live specs from the repo, ask one question per section
3 to 4	Edit specs for known features	Pair edit with an engineer, ship one merged spec change
5 to 6	Author a small spec	Write the spec for one real ticket, get it through review
7 to 8	Review others	Approve or reject five spec PRs with written reasoning
9 to 10	Lead a spec session	Run a spec authoring workshop for the next cohort
11 to 12	Coach	Pair with a new joiner for two weeks, hand them Level 1

COMMON STUMBLING BLOCKS BY ROLE

ROLE	WHAT TRIPS THEM UP	WHAT UNBLOCKS THEM
Developer	Trust gap with AI generated code	Pair on small spec, ship one win, then scale
Product Owner	Writes narratives instead of intent	Templates with intent, constraint, verification fields
QA	Used to test plans, not verification blocks	Convert one existing test plan into a verification block
EM	Measures velocity, not spec quality	New scorecard with spec clarity and acceptance rate
Business stakeholder	Specifies solutions, not problems	Spec prompt asks "what changes for the user" first

Sample Spec: AI Standup Assistant

What a working spec actually looks like. Compact, every section present. Goal here is shape, not exhaustive detail. Notice that nothing in this file is in Jira, in Confluence, or in someone's head.

SPEC.md · v1.2 · owners: @engineering, @product

AI Standup Assistant

1. GOAL

Retire the synchronous daily standup for a 12 person engineering squad. Each member receives a Slack DM at 09:00 local, answers three async questions, and the bot composes a team summary posted to the squad channel at 10:00.

2. NON GOALS

- Not a project management tool
- Not a 1:1 coaching surface
- No video, audio, or transcription support
- No integration with Jira in v1

3. USERS AND STAKEHOLDERS

- Engineers (12, primary users, daily interaction)
- Engineering Manager (consumer of the summary)
- Product Owner (read access to summary channel)

4. CONSTRAINTS

- Slack only, must work on Slack Free tier
- Three timezones: London, Bangalore, San Francisco
- No PII storage beyond Slack user ID
- p95 summary generation under 30 seconds
- Monthly LLM spend under £50

5. ARCHITECTURE DECISIONS

- LLM: Claude Sonnet for summary. Auto fallback to Haiku if monthly spend exceeds £40
- Storage: Postgres on Fly.io, 90 day retention, then anonymise
- Deployment: single container, no orchestration
- No queue, direct invocation is acceptable at 12 users
- Scheduler: cron in app, not external service, to keep ops surface small

6. DATA MODEL

```
StandupResponse {
  id: uuid
  user_id: string         // Slack user ID
  date: date
  answers: { yesterday, today, blockers }
  submitted_at: timestamp
}

DailySummary {
  id: uuid
  squad_id: string
  date: date
  body: text
  posted_at: timestamp
}
```

7. INTERFACES

- Slack interactive DM at 09:00 local with a modal form
- Slash command `/standup` for manual submission
- Channel post at 10:00 local with the composed summary
- Internal endpoint `POST /admin/regenerate` for EM override

8. BEHAVIOURS

- **B1.** Engineer receives DM at 09:00 in their local timezone
- **B2.** Engineer submits via Slack modal or `/standup`
- **B3.** Bot composes summary at 10:00 local even if responses are incomplete
- **B4.** Blockers section appears in the summary even if only one engineer reports one

- **B5.** If an engineer does not respond for three consecutive days, the bot DMs the EM

9. VERIFICATION BLOCKS

> **V1.** Given 09:00 in user timezone, when scheduler fires, then DM is sent within 60 seconds

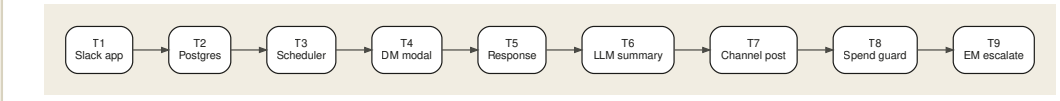
> **V2.** Given engineer submits at 09:45, when 10:00 fires, then their response appears in the summary

> **V3.** Given zero responses received, when 10:00 fires, then summary states "no responses today" instead of failing

> **V4.** Given monthly LLM spend exceeds £40, when next summary runs, then provider switches to Haiku and posts a notice in #eng-ops

> **V5.** Given an engineer has missed three days, when day three summary runs, then EM receives a DM with the user ID

10. TASK GRAPH



11. OPEN QUESTIONS

- Do we want sentiment tracking in v2 or is that out of scope by design
- Should holidays auto pause the bot per user calendar
- Is anonymous mode required for any team

12. ACCEPTANCE

- V1 through V5 pass in integration tests against a sandbox Slack workspace

- EM approves three consecutive daily summaries without manual edits
- Engineering squad votes to retire the synchronous standup at sprint 14

13. VERSION HISTORY

```
v1.0 · 2026-04-12 · initial draft
v1.1 · 2026-04-19 · added timezone handling after pilot feedback
v1.2 · 2026-04-26 · added B4 blocker behaviour and V4 spend guardrail
```

Around 400 words. The AI reads it, picks up the task graph, starts work. When the spec changes, the build follows.

Building the Spec with Claude

Taking the standup spec above into Claude Code. Same spec, three files at the repo root, and the build loop runs itself.

REPOSITORY SETUP

```
// repo root
SPEC.md                  // the standup assistant spec from above
CLAUDE.md                // build rules and pointers
memory/
  architecture.md        // stable decisions across sessions
  decisions.md
src/
```

CLAUDE.MD, THE ENTRY POINT

```
## Build Rules
1. Read SPEC.md before any code change
2. Every commit references a spec section (B1, V1, T1)
3. Tests follow the V block ID convention
4. Propose spec edits via PR before implementation

## Active Spec
SPEC.md v1.2, AI Standup Assistant

## Current Task
T3 Scheduler cron, satisfies B1 and V1

## Memory
See memory/architecture.md and memory/decisions.md
```

A REAL SESSION

Prompt: "*Implement T3 from the task graph. Must satisfy B1 and V1.*"

Claude reads SPEC.md, locates T3, B1, V1. Proposes a node cron scheduler with timezone handling. Writes `v1_dm_sent_within_60s` test against a Slack mock. Commits with message `T3 scheduler cron, satisfies B1 / V1`. Updates memory/decisions.md with the cron choice and why.

Two days later, B5 escalation needs adding. Claude opens a spec PR first, increments to v1.3, gets approval, then implements. The spec leads, the code follows.

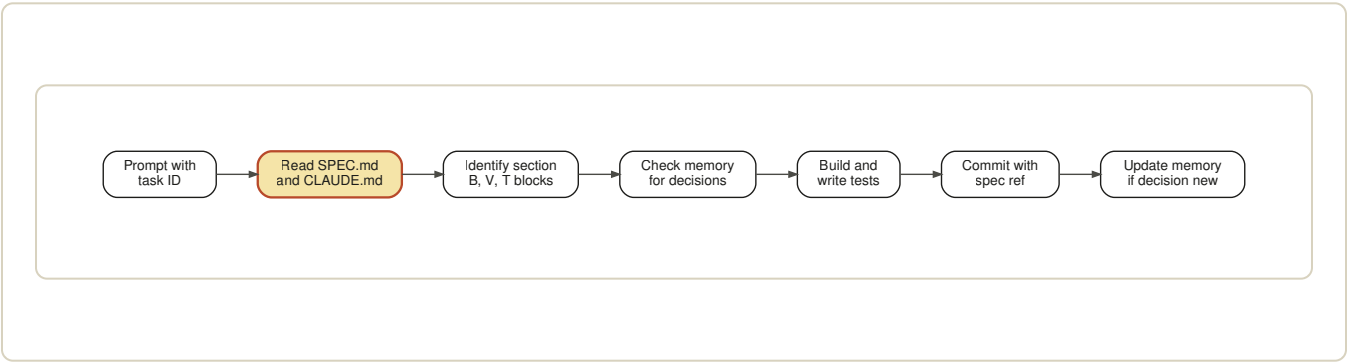


Figure 9. The Claude build loop. Spec in, traceable diff out.

WHY THIS WORKS IN CLAUDE

PARAMETER	GENERIC AI TOOL	CLAUDE CODE ON SDD
Context loading	Manual paste each session	CLAUDE.md and SPEC.md read every session
Multi step execution	One prompt, one diff	Native agentic loop across the task graph
Cross session memory	None, restart from zero	Memory directory survives sessions
Task tracking	You hold it in your head	Built in task tool tied to spec sections
Spec edits	Manual rewrite	AI proposes spec diff with rationale

One rule that makes or breaks the setup. Keep CLAUDE.md under 100 lines. It loads on every session. Long files become noise. The spec carries the detail. CLAUDE.md just points at it.

Closing Thought

AIDLC is not Agile with AI bolted on the side. It is a different operating model. The spec replaces the standup as the heartbeat of delivery. Stakeholders who learn to write specs gain real leverage in how systems get built. The ones who treat AIDLC as a tooling decision will get the same outcomes they got from Agile, only with a faster cycle time and a bigger AI bill.
